

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

Universidad Tecnológica Nacional

Programación 1

Trabajo Práctico Integrador

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Integrante: Torres, Rubén Alejandro

Fecha de entrega: 29 de junio de 2026

Repositorio: <https://github.com/aics11/-Gesti-n-de-Datos-de-Pa-ses-en-Python-filtros-ordenamientos-y-estad-sticas.git>

Video: [\[TU LINK DE VIDEO\]](#)

Índice

1. Marco Teórico	3
2. Decisiones Técnicas y Arquitectura	6
3. Dificultades y Conclusiones	8
4. Bibliografía	9

1. Marco Teórico

Para realizar este trabajo se retomaron varios contenidos vistos en Programación 1 y se los aplicó en un sistema sencillo de gestión de datos de países. En las siguientes secciones se explican esos conceptos y, sobre todo, cómo se usaron dentro del programa desarrollado.

1.1 Listas

Una lista en Python es una estructura de datos que permite almacenar una colección ordenada de elementos. A diferencia de otros lenguajes, las listas en Python son dinámicas: pueden crecer o reducirse en tiempo de ejecución y pueden contener elementos de distintos tipos.

En nuestro programa, la lista `países[]` es el punto de partida para casi todas las operaciones. Allí se guardan los países cargados desde el archivo y, a partir de esa lista, se realizan búsquedas, filtros, ordenamientos y cálculos estadísticos.

*Fuente: Python Software Foundation. (2024). Python Docs - Lists.
<https://docs.python.org/3/tutorial/introduction.html#lists>*

1.2 Diccionarios

Un diccionario en Python es una estructura de datos que almacena pares clave-valor. Cada clave es única dentro del diccionario y permite acceder a su valor de forma directa y eficiente.

Para representar cada país se utilizó un diccionario con los datos principales: nombre, población, superficie y continente. Esta forma de organizar la información resultó práctica porque permite acceder a cada dato por su clave, por ejemplo `país['nombre']` para consultar el nombre o `país['población']` para obtener la cantidad de habitantes.

*Fuente: Python Software Foundation. (2024). Python Docs - Dictionaries.
<https://docs.python.org/3/tutorial/datastructures.html#dictionaries>*

1.3 Funciones

Una función es un bloque de código reutilizable que realiza una tarea específica. En Python se definen con la palabra clave `def`. El uso de funciones permite modularizar el código, lo cual facilita su lectura, mantenimiento y corrección de errores.

En el código se decidió separar las acciones principales en funciones, como `cargar_csv()`, `agregar_pais()`, `buscar_pais()`, `filtrar_paises()`, `ordenar_paises()` y `mostrar_estadisticas()`. Esta organización ayudó a que el programa fuera más fácil de leer y también facilitó probar cada parte por separado.

Fuente: Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.

1.4 Condicionales

Las estructuras condicionales (`if`, `elif`, `else`) permiten ejecutar diferentes bloques de código según se cumpla o no una condición. Son fundamentales para validar datos de entrada y tomar decisiones en el flujo del programa.

Las condicionales aparecen en varias partes del programa, especialmente para validar los datos ingresados y evitar errores. Por ejemplo, se usan para controlar que el nombre no quede vacío, que la población no sea negativa, que no se repita un

país ya cargado y que se muestre un mensaje adecuado cuando una búsqueda no encuentra resultados.

1.5 Ordenamientos

El ordenamiento es el proceso de reorganizar una colección de elementos según un criterio determinado. El algoritmo de burbuja (bubble sort) es uno de los más simples: compara elementos adyacentes y los intercambia si están en el orden incorrecto, repitiendo el proceso hasta que la lista quede ordenada.

Como parte de la consigna, el ordenamiento se resolvió programando el algoritmo de burbuja de forma manual, sin recurrir a `sorted()` ni a `.sort()`. Con este procedimiento se pueden ordenar los países por nombre, población o superficie, en sentido ascendente o descendente según lo que elija el usuario.

Fuente: Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

1.6 Estadísticas básicas

La parte de estadísticas permite obtener una mirada general sobre los datos cargados. En este caso se calcularon resultados como el país con mayor y menor población, los promedios de población y superficie, y la cantidad de países por continente. Para obtener esos valores fue necesario recorrer la lista y acumular la información correspondiente.

1.7 Archivos CSV

Un archivo CSV (Comma-Separated Values) es un formato de texto plano donde los datos están separados por comas. Es ampliamente utilizado para intercambiar datos entre aplicaciones porque es simple, liviano y legible.

El archivo `países.csv` se usó como base de datos inicial del programa. Al iniciar la ejecución, la función `cargar_csv()` lee ese archivo con el módulo `csv` de Python, convierte los valores numéricos al tipo entero y guarda cada país dentro de la lista `países[]`.

Fuente: Python Software Foundation. (2024). Python Docs - csv module. <https://docs.python.org/3/library/csv.html>

2. Decisiones Técnicas y Arquitectura

2.1 Estructura de datos elegida

La estructura elegida fue una lista de diccionarios porque se adapta bien al tipo de información que se necesitaba manejar. Cada país queda guardado como un conjunto de datos relacionados entre sí: nombre, población, superficie y continente. Para el alcance del trabajo, esta solución fue suficiente y permitió implementar las búsquedas, filtros, ordenamientos y estadísticas sin agregar una complejidad innecesaria.

2.2 Modularización

Para mantener el código ordenado, se dividió el programa en funciones. Esto hizo que cada parte tuviera una tarea concreta y que fuera más sencillo detectar errores o modificar alguna funcionalidad sin afectar todo el sistema:

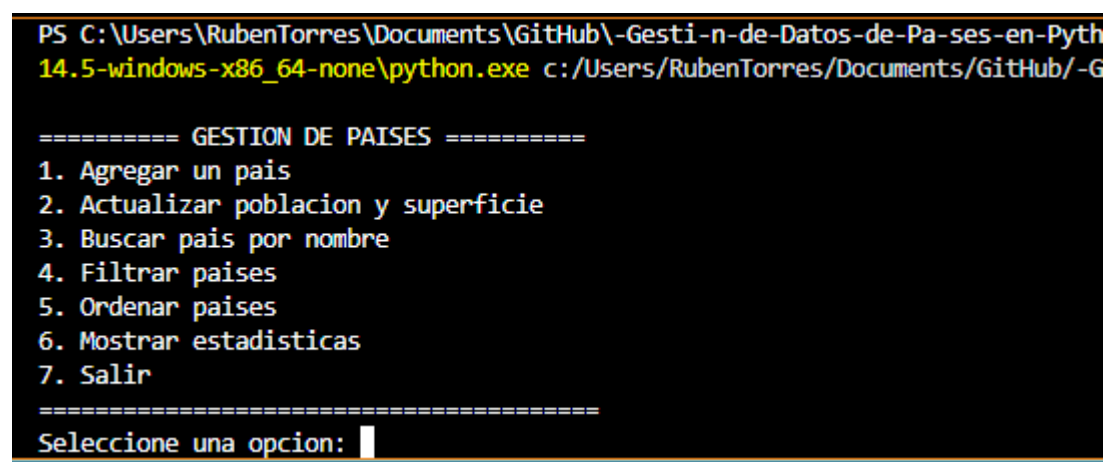
- `cargar_csv()`: lectura del archivo CSV y carga inicial de datos
- `agregar_pais()`: validación y alta de un nuevo país
- `actualizar_pais()`: modificación de población y superficie
- `buscar_pais()`: búsqueda por nombre con coincidencia parcial
- `filtrar_paises()`: filtrado por continente, población o superficie
- `ordenar_paises()`: ordenamiento por burbuja en múltiples criterios
- `mostrar_estadisticas()`: cálculo de indicadores generales
- `mostrar_menu()`: presentación del menú principal

2.3 Flujo general del programa

El funcionamiento general es simple: primero se cargan los datos desde el archivo CSV y luego se muestra un menú de opciones. A partir de la elección del usuario, el programa llama a la función correspondiente y trabaja con la lista de países. Este proceso se repite hasta que se selecciona la opción de salir.

2.4 Capturas de pantalla

1.- El menú principal



```
PS C:\Users\RubenTorres\Documents\GitHub\Gesti-n-de-Datos-de-Pa-ses-en-Pyth 14.5-windows-x86_64-none\python.exe c:/Users/RubenTorres/Documents/GitHub/-G

=====  
GESTION DE PAISES  
=====
1. Agregar un pais
2. Actualizar poblacion y superficie
3. Buscar pais por nombre
4. Filtrar paises
5. Ordenar paises
6. Mostrar estadisticas
7. Salir
=====
Seleccione una opcion: |
```

2.-Carga del CSV exitosa + menú

```

7.- Suma
=====
Seleccione una opcion: 1

--- Agregar Pais ---
Nombre del pais: Cabo Verde
Poblacion: 530000
Superficie en km2: 4033
Continente: África
Pais 'Cabo Verde' agregado correctamente.

```

3.- Buscar un país (caso exitoso)

```

=====
Seleccione una opcion: 3

--- Buscar Pais ---
Ingrese el nombre o parte del nombre: arg

Se encontraron 1 resultado(s):
  Argentina | Poblacion: 45,376,763 | Superficie: 2,780,400 km2 | Continente: América

```

4.- Buscar un país (caso sin resultados)

5.- Filtrar por continente

6.- Ordenar por población descendente

```

--- Ordenar Paises ---
1. Por nombre
2. Por poblacion
3. Por superficie
Seleccione criterio: 1
Orden: 'A' ascendente o 'D' descendente: d

Paises ordenados:
  Sudáfrica | Poblacion: 59,308,690 | Superficie: 1,219,090 km2 | Continente: África
  Rusia | Poblacion: 145,934,462 | Superficie: 17,098,242 km2 | Continente: Europa
  Reino Unido | Poblacion: 67,886,011 | Superficie: 243,610 km2 | Continente: Europa
  Nueva Zelanda | Poblacion: 5,002,100 | Superficie: 270,467 km2 | Continente: Oceanía

```

7.- Estadísticas

```

=====
Seleccione una opcion: 6

--- Estadísticas ---
Total de paises: 25
Pais con mayor poblacion: China (1,439,323,776)
Pais con menor poblacion: Cabo Verde (530,000)
Promedio de poblacion: 203,366,746
Promedio de superficie: 3,260,703 km2

Paises por continente:
  América: 7 paises
  Asia: 5 paises
  Europa: 6 paises
  África: 5 paises
  Oceanía: 2 paises

```

8.- Agregar un país (validación de error)

```
=====
Seleccione una opcion: 1
```

```
--- Agregar Pais ---
```

```
Nombre del pais:
```

```
Error: el nombre no puede estar vacio.
```

3. Dificultades y Conclusiones

3.1 Dificultades encontradas

Una de las dificultades principales fue hacer que el ordenamiento por burbuja funcionara correctamente en distintos casos. No solo debía ordenar por nombre, población o superficie, sino también permitir el orden ascendente y descendente. Para resolverlo, se utilizó una variable booleana que indica la dirección del ordenamiento y simplifica la comparación entre elementos.

También fue necesario prestar atención a la carga del archivo CSV. Los datos numéricos se leen como texto, por lo que hubo que convertir la población y la superficie a enteros antes de compararlos o usarlos en cálculos. Para evitar que un dato mal escrito detuviera todo el programa, se incorporó un bloque try/except durante la lectura del archivo.

3.2 Conclusiones

A través de este proyecto se pudieron poner en práctica varios temas trabajados durante la materia, como listas, diccionarios, funciones, condicionales, bucles, manejo de archivos y algoritmos de ordenamiento.

Además, el trabajo permitió comprobar la utilidad de dividir el código en partes más pequeñas. Al tener funciones separadas, resulta más sencillo leer el programa, encontrar errores y agregar mejoras. El uso de try/except también fue importante, ya que ayuda a que el sistema siga funcionando, aunque aparezcan entradas inesperadas o datos con formato incorrecto.

En conclusión, el sistema muestra que con estructuras simples y una organización clara del código se puede construir una aplicación útil para cargar, consultar, ordenar y analizar datos. Aunque se trata de un proyecto inicial, deja una buena base para futuras mejoras.

4. Bibliografía

Python Software Foundation. (2024). The Python Tutorial.

<https://docs.python.org/3/tutorial/>

Python Software Foundation. (2024). csv — CSV File Reading and Writing.

<https://docs.python.org/3/library/csv.html>

Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

Guttag, J. V. (2021). Introduction to Computation and Programming Using Python (3rd ed.). MIT Press.